



Degree Project in Technology

First Cycle, 15 credits

Hybrid Modelling Approach to Optimise the Train Speed Profile

Is machine learning a good way to optimize
train driving

Joel Håkansson & Adrian Nagorka

Abstract

Optimising train speed profiles (i.e. a graph with position on the X axis and speed on the Y axis) is a challenging task essential for improving energy efficiency and punctuality in rail operations. Traditional optimisation approaches either rely on rigid physics-based models or purely data-driven methods, each with limitations. This thesis explores a hybrid modelling approach that combines physics-informed constraints with machine learning, leveraging a Soft Actor-Critic (SAC) reinforcement learning agent interacting with a realistic train simulation (Open Rails). The aim is to generate train driving strategies that adhere to operational constraints (speed limits, schedules) while minimising energy usage. We present background on reinforcement learning applications in transportation, detail our methodology for integrating SAC with the Open Rails environment, and discuss the training process and outcomes across multiple iterations of the agent. The results demonstrate that the hybrid approach can produce feasible speed profiles that satisfy safety and timing requirements. We analyse the agent’s learned policy, highlight limitations in reward tuning and training time, and suggest improvements for future work, including parallel training and enhanced state representations.

Sammanfattning

Att optimera tågens hastighetsprofiler (dvs. ett diagram med position på X-axeln och hastighet på Y-axeln) är en utmanande uppgift som är avgörande för att förbättra energieffektivitet och punktlighet i järnvägsdrift. Traditionella optimeringsmetoder bygger antingen på rigida fysikbaserade modeller eller rent data-drivna metoder, vilka båda har sina begränsningar. Denna uppsats undersöker en hybridmodelleringsmetod som kombinerar fysikinformerade begränsningar med maskininlärning, genom att använda en förstärkningsinlärningsagent av typen Soft Actor-Critic (SAC) som interagerar med en realistisk tågsimulering (Open Rails). Syftet är att generera körstrategier för tåg som följer operativa begränsningar (hastighetsbegränsningar, tidtabeller) samtidigt som energiförbrukningen minimeras. Vi presenterar bakgrund om förstärkningsinlärnings tillämpningar inom transport, redogör för vår metodik för att integrera SAC med Open Rails-miljön, samt diskuterar träningsprocessen och resultaten från flera iterationer av agenten. Resultaten visar att den hybridbaserade metoden kan generera genomförbara hastighetsprofiler som uppfyller säkerhets- och tidskrav. Vi analyserar agentens inlärd policy, lyfter fram begränsningar i belöningsjustering och träningstid, och föreslår förbättringar för framtida arbete, inklusive parallell träning och förbättrade tillståndsrepresentationer.

Acknowledgements

We would like to thank our supervisor, Yuhua Yao as she has helped us throughout the entire process and answered our various questions. She has been a great help to keep the project on track despite the various problems.

Contents

1	Introduction	6
1.1	Context and Motivation	6
1.2	Problem Statement and Goals	6
2	Background and Literature Review	8
2.1	Reinforcement Learning in Transportation	8
2.1.1	General Transportation Context	8
2.1.2	RL in Rail Transportation	8
2.1.3	Methodological Approaches	9
2.2	SAC – Soft Actor-Critic	9
2.3	Summary and Gap Analysis	11
3	Methodology	13
3.1	Reinforcement Learning Setup	13
3.2	Parallelisation or Single-Instance Training	14
3.3	Soft Actor-Critic Algorithm and Open Rails Simulator	15
3.3.1	Mathematical Formulation	15
3.3.2	Implementation in Code	16
3.3.3	Changes in the Agent	20
3.4	Route Description and Scenario	23
3.5	Human Driving Profile (Baseline)	24
3.6	Hardware and Runtime Setup	25
4	Results	27
4.1	Training Curves	27
4.2	Energy Consumption Analysis	29
4.3	Schedule Adherence	30
4.4	Driving Profiles	31

5	Discussion	36
5.1	Interpretation of Learned Policy	36
5.2	Limitations	37
5.3	Implications	39
6	Conclusion	41
6.1	Summary of Findings	41
6.2	Contributions	41
6.3	Future Work	42
7	Appendix	46

1 Introduction

1.1 Context and Motivation

This project is a Bachelor’s thesis within the Optimisation branch of the Department of Mathematics at KTH Royal Institute of Technology in Sweden. The theme is hybrid modelling, specifically applied to optimising train speed profiles (i.e. a graph with position on the X axis and speed on the Y axis). The motivation behind this project is twofold: first, to experiment with machine learning and its implementation in real-world systems; second, to utilise a model (train speed profiles) whose optimisation success can be intuitively evaluated. Trains may appear simple initially but involve numerous interconnected variables. While the basic physics and dynamics of train motion are familiar, optimising train speed profiles is complex using traditional methods of doing physics based optimizing by hand. Thus, this project explores whether machine learning techniques can achieve similar or superior optimisation outcomes when combined with physical insights.

Optimising train speed profiles is a critical task in modern railway operations, aimed at improving energy efficiency, reducing travel time, and ensuring safety and passenger comfort. Traditional approaches to speed optimisation often rely on either purely physics-based models or purely data-driven methods. However, each of these approaches has limitations: while physics-based models can lack flexibility in handling real-world variability, data-driven models may struggle with generalisation and interpretability.

In this context, hybrid modelling offers a promising alternative by combining the strengths of both paradigms. By integrating physics-informed constraints with machine learning techniques, hybrid models can accurately capture the non-linear dynamics of train movement while adapting to operational data. Such approaches are especially valuable for complex railway systems where factors such as gradient, curvature, station schedules, and energy usage interact in nontrivial ways.

1.2 Problem Statement and Goals

The project aims to optimise the speed profile of a train travelling between a given origin and destination (station A and station B) through hybrid modelling. The speed profile, specifying speed and timing along the route, is critical for both punctuality and energy efficiency. Key optimisation criteria

include adhering to schedule constraints and maintaining compliance with track speed limits.

Our approach employs a reinforcement learning (RL) agent to derive optimal speed profiles. The RL agent’s performance is evaluated through a reward function that balances multiple objectives (mainly timely arrival and speed limit compliance). The raw RL-generated speed profiles are then validated and refined by incorporating physical constraints (train dynamics and traction characteristics) to ensure practical feasibility and efficiency.

Simulation and data collection are conducted via *Open Rails*, a free, open-source train simulation platform. The RL agent interacts with the simulator to iteratively learn optimal driving decisions based on environmental feedback, continuing training until its performance converges to a satisfactory level.

2 Background and Literature Review

2.1 Reinforcement Learning in Transportation

Reinforcement learning has been applied across various transportation domains to improve operational efficiency and decision-making. In general transportation contexts, RL techniques have been explored for problems such as traffic signal control, dynamic routing and scheduling, and autonomous vehicle driving strategies [1]. These applications demonstrate RL’s ability to handle complex, stochastic environments and learn policies that adapt to real-time conditions.

2.1.1 General Transportation Context

Within road and urban transport, for example, RL has been used to optimise traffic light timings to reduce congestion and delays, as well as to develop adaptive cruise control and routing strategies for vehicles [1]. Such studies highlight how agents can learn to make real-time decisions (e.g., adjusting signal phases or vehicle speed) that improve throughput or safety compared to static rule-based systems. The success of RL in these areas motivates its exploration in rail systems, which have their own set of challenges and constraints.

2.1.2 RL in Rail Transportation

In the railway domain, RL has been investigated for several key operational tasks. Significant works include Li and Ni [2], who proposed multi-agent RL for dynamic train timetabling, and Ying et al. [3], who applied an actor-critic method for metro train scheduling under stochastic demand. Broadly, RL in rail has been applied to:

- **Train Timetable Rescheduling and Dynamic Operations:** RL agents have been used to dynamically adjust train schedules in response to disturbances such as delays or disruptions, aiming to optimise service reliability and overall network efficiency [1]. For instance, Li and Ni’s framework [2] learns to make rescheduling decisions that minimise delay propagation and enable real-time single-track dispatching by combining Q-learning with graph search methods [4].

- **Energy Management and Efficiency:** RL algorithms have been employed in train driving strategies to minimise energy consumption. Classical optimal-control formulations remain highly competitive and are still used as baselines [5]. Techniques like Q-learning and actor-critic methods facilitate efficient throttle and braking control to achieve smoother and more energy-efficient operations. Multi-step RL approaches (e.g., Zhou et al. [6]) have shown that predictive energy management can be improved without relying on additional Markov models, demonstrating RL’s versatility in optimising energy usage [6, 7].
- **Autonomous Train Operation (ATO):** RL supports the development of increasingly autonomous train control systems by continuously learning optimal speed and braking control policies. Studies have addressed path-following and collision avoidance using RL, contributing to the robustness of autonomous train operations. For example, Martinsen and Lekkas [8] demonstrated how deep RL can handle complex operational scenarios, improving safety and precision in fully automated train driving, and recent work has extended these ideas to urban-rail ATO controllers based on deep RL [9].

2.1.3 Methodological Approaches

The reinforcement learning methodologies explored in rail transport primarily involve:

- **Value-Based Methods** (e.g., Q-learning), which optimise decisions in discrete action spaces.
- **Policy-Based and Actor-Critic Methods** (e.g., Proximal Policy Optimisation (PPO), Deep Deterministic Policy Gradient (DDPG)), which excel in continuous and high-dimensional control tasks typical of dynamic train operations.

In our work, we employ an actor-critic method (Soft Actor-Critic) due to its advantages in continuous action domains.

2.2 SAC – Soft Actor-Critic

Soft Actor-Critic (SAC) is a model-free, off-policy, actor-critic RL algorithm designed for continuous control problems. It learns to take actions (such as

adjusting throttle or braking) that maximise not only a cumulative reward but also an entropy term, effectively encouraging exploration. At its core, SAC simultaneously employs:

1. A **Critic** function that estimates the value (expected return) of state-action pairs.
2. An **Actor** (policy) that outputs a stochastic action distribution for each state.

What makes SAC unique among RL algorithms is that it encourages the policy to remain stochastic (i.e., somewhat random) to ensure continuous exploration and avoid getting trapped in suboptimal strategies.

Being off-policy, SAC can reuse experience data from past interactions. Unlike on-policy methods that require fresh trajectories for each policy update, an off-policy algorithm like SAC can learn from a replay buffer of experiences collected under earlier policies. This greatly improves sample efficiency, meaning the agent can learn more from each interaction.

The main advantages of SAC are:

- **Stable Learning:** SAC uses two independent critic networks and periodically updated target networks, which help prevent training instability and overestimation bias: common issues in deep RL.
- **Efficient Exploration:** The entropy bonus in SAC’s objective ensures the agent continuously tries new strategies. This helps the agent discover better solutions in environments where rewards may be sparse or delayed.
- **Sample Efficiency:** As an off-policy learner, SAC reuses past experience for training. This allows it to make better use of each simulator interaction, a crucial property when simulations are time-consuming.
- **Effectiveness in Continuous Action Spaces:** SAC is well-suited to tasks like driving or train control, where actions (throttle/brake levels) are continuous rather than discrete.

In real-world contexts, SAC has shown promise. For instance, Pei and Lan [10] developed a high-speed train control strategy based on SAC that improved operational efficiency and energy use. Beyond rail, the creators

of SAC (Haarnoja et al. [11]) demonstrated its effectiveness on challenging robotic control tasks (e.g., locomotion and manipulation), achieving state-of-the-art performance with improved stability and reduced training time compared to earlier algorithms. These successes underscore SAC’s capability to handle continuous control and motivate our choice of SAC for this project.

2.3 Summary and Gap Analysis

Recent research demonstrates effective energy-efficient train operations using both classical optimal control techniques (e.g., based on Pontryagin’s maximum principle and dynamic programming) and more adaptive RL approaches like SAC. For example, classical methods have been well documented in studies such as Khmelnitsky’s optimal train control formulation [12], while recent RL-based methods have shown promising real-time adaptability in train control [10, 13].

Despite these advances, several gaps remain:

- **Computational Efficiency and Real-Time Adaptability:** Classical optimal control methods can produce theoretically optimal solutions, but their high computational demand hinders real-time application. Conversely, many RL strategies require extensive offline training and optimise broader objectives, which can limit their ability to make instantaneous adjustments to disturbances [10]. There is a need for approaches that maintain near-optimal control performance while being lightweight enough for real-time use.
- **Direct Focus on Speed Profile Optimisation:** Many studies (e.g., addressing energy efficiency and safety [13, 14]) treat the train’s speed trajectory as a secondary outcome of optimisation. In those cases, the speed profile emerges as a byproduct of solving a larger control problem, rather than being optimised directly.

Project Focus: This project directly addresses the above gaps by prioritising the production of an optimal speed curve as the main objective. By integrating detailed train dynamics into a real-time RL framework (using SAC), our method aims to reduce computational overhead and enhance adaptability under disturbances. We focus the learning agent on the core problem of speed profile optimisation under operational constraints, rather than diluting its objective with too many competing criteria. This approach is expected to

yield a more efficient driving profile, effectively filling the gap left by broader control-oriented methods.

Furthermore, by combining physics-based elements (through the simulator and constraint enforcement) with learning, we leverage the strengths of both worlds. This hybrid strategy provides a balance between theoretical optimality and practical adaptability, making progress toward an approach that could be deployed in real train operations.

3 Methodology

3.1 Reinforcement Learning Setup

Our implementation of the reinforcement learning solution is done in Python using Ray RLlib, a library for scalable RL. The core components of the RL setup are the environment, the agent (policy), and the reward structure:

- **Environment:** The environment serves as the interface between the RL algorithm and the train simulation. It is responsible for sending control instructions to the simulator and returning observations (state information) along with rewards. In our case, the environment wraps the Open Rails simulator, exposing the train’s state (such as current speed, remaining distance to the next stop, etc.) to the RL agent and applying throttle or brake commands from the agent to the simulation.
- **Agent:** The agent (or actor) issues actions and receives observations and rewards in return. Through trial and error, the agent learns a policy that chooses actions to maximise cumulative reward. In our project, the agent’s actions consist of setting the throttle and brake levels.
- **Reward Function:** The reward function is carefully designed to guide the agent towards the desired objectives. It provides positive or negative feedback at each time step based on the agent’s behaviour – for example, penalties for exceeding speed limits or arriving late.

A key part of the environment is its communication with the simulation software. Open Rails provides a built-in API accessible via the local machine’s network ports. We modified the Open Rails source slightly to ensure we could send commands (throttle/brake) and retrieve state information through this API at each simulation tick. This design allows the RL agent to interact with the simulator in real time: at each decision step, the agent sends a throttle/brake command, the simulator advances the train and returns an updated state, and the environment computes the reward corresponding to that transition.

The action space for the agent is defined to reflect continuous control of throttle and brakes. We represent the action as a two-dimensional continuous vector: the first component is the throttle setting and the second is the

brake pressure. Both range from 0 (no throttle or brake) to 1 (full throttle or full service brake). In practice, we restrict the effective brake range such that 1 corresponds to a moderate braking force (50% of maximum, and later further reduced to 33%) because applying emergency-level braking is typically unnecessary except for extreme scenarios and would only lead to inefficient driving. By capping the brake action, we encourage the agent to find solutions without resorting to overly harsh braking, which aligns with how human drivers operate.

During each simulation step, relevant physical data (speed, acceleration, distance to upcoming speed restrictions or stations, etc.) are logged by Open Rails. Our environment reads these logs (or receives them via the API) to form the observation that is given to the agent. Based on this observation, the agent decides on the next throttle/brake action. The loop then repeats: the action is applied, the simulation updates, and a new observation and reward are obtained. This closed-loop interaction enables the agent to gradually learn an optimal driving strategy.

3.2 Parallelisation or Single-Instance Training

In reinforcement learning, using multiple environment instances in parallel can speed up training by collecting more experience data in the same wall-clock time. We considered both parallel and single-instance training models for our SAC agent. Running several Open Rails simulations concurrently would allow the agent to explore more trajectories per training iteration, improving the diversity of experience.

However, in practice we faced significant constraints in parallelising the environment. Each Open Rails simulation while not computationally heavy, requires a separate instance of the game engine running. Due to the program’s age it is prone to crashing or running into errors. This behaviour is amplified when more than one instance is running at a time. Coordinating multiple instances also involved assigning unique communication ports for each (a functionality we implemented by offsetting the base port for each worker). Ultimately, due to stability issues when running multiple instances of Open Rails simultaneously, we opted to use a single environment instance for most training. In our configuration, we set RLlib’s number of parallel workers to 1, meaning the training loop interacts with one simulated train at a time.

It is worth noting that our code is structured to allow parallel execution

in future experiments. The environment registration in Ray RLlib assigns each worker a different port number for Open Rails, so scaling up to (for example) two parallel simulator instances would simply require launching two Open Rails sessions on separate ports. This could greatly accelerate training if sufficient computational resources are available, as discussed in part 6.2 "Future work". For the scope of this project, though, a single-instance approach was the most practical and ensured the simulation and agent remained synchronised without overloading our systems.

3.3 Soft Actor-Critic Algorithm and Open Rails Simulator

This section details how the SAC algorithm is applied to our train control problem and how it is integrated with the Open Rails simulator.

3.3.1 Mathematical Formulation

The primary goal of the SAC agent is to maximise a combined objective consisting of expected cumulative rewards and policy entropy. Mathematically, the objective can be expressed as:

$$J(\pi) = \mathbb{E}_{(s_t, a_t) \sim \rho_\pi} \left[\sum_{t=0}^T (r(s_t, a_t) + \alpha H(\pi(\cdot | s_t))) \right], \quad (1)$$

where $r(s_t, a_t)$ is the immediate reward for taking action a_t in state s_t , $\pi(\cdot | s_t)$ is the stochastic policy (actor) giving a distribution over actions in state s_t , $H(\pi(\cdot | s_t))$ is the entropy of the policy's action distribution at s_t , and α is the temperature parameter that controls the trade-off between reward maximisation and entropy (exploration encouragement). A higher α places more emphasis on entropy (exploration), while a lower α focuses the agent more on exploiting what it already knows to gain reward.

SAC uses two kinds of neural network function approximators:

- **Critic Networks (Q -functions):** These estimate the expected cumulative reward (return) for given state-action pairs. We use two critic networks $Q_{\theta_1}(s, a)$ and $Q_{\theta_2}(s, a)$ with parameters θ_1, θ_2 to mitigate positive bias in value estimates. The critics are trained by minimising the loss:

$$L_{\text{critic}}(\theta_i) = \mathbb{E}_{(s, a, r, s') \sim D} \left[(Q_{\theta_i}(s, a) - y)^2 \right], \quad (2)$$

where the target y is given by

$$y = r + \gamma \mathbb{E}_{a' \sim \pi} \left[\min_{j=1,2} Q_{\bar{\theta}_j}(s', a') - \alpha \log \pi(a' | s') \right].$$

Here s' is the next state, γ is the discount factor, and $Q_{\bar{\theta}_j}$ are target networks (periodically updated copies of the critic networks) used to stabilise training. The target y represents the current estimate of the “Q-value” backed up from the next state s' .

- **Actor Network (Policy π_ϕ):** This network outputs the parameters of a stochastic policy $\pi_\phi(a | s)$ (in our case, mean and variance for Gaussian distributions over continuous throttle and brake actions). The actor is trained to maximise a reward-plus-entropy objective, which is equivalent to minimising the following:

$$L_{\text{actor}}(\phi) = \mathbb{E}_{s \sim D} \left[\mathbb{E}_{a \sim \pi_\phi(\cdot | s)} [\alpha \log \pi_\phi(a | s) - Q_\theta(s, a)] \right], \quad (3)$$

where $Q_\theta(s, a)$ is the value from one of the critic networks (we use the minimum of the two critics for stability). This objective encourages the policy to choose actions that have both low Q -value penalty (i.e., high expected return) and high entropy (exploration). Intuitively, the policy learns to avoid actions the critics judge as poor, while still exploring to find better actions.

3.3.2 Implementation in Code

Our implementation utilises RLlib’s built-in SAC algorithm, configured for our custom environment. We register the Open Rails environment with RLlib by providing an environment constructor function that instantiates our `OpenRailsEnv` class and sets it up with the appropriate communication port. Once registered (under ID `"OpenRails-v0"`), RLlib’s trainer can create environment instances as needed.

The training configuration for SAC is carefully tuned. Key hyperparameters and settings include:

- **Discount factor (γ):** We use $\gamma = 0.995$ to value future rewards highly (since the episode length, corresponding to a journey between stations, is relatively long).

- **Neural network architecture:** Both the policy and critic networks are feed-forward neural networks with two hidden layers of 256 units each (with ReLU activations). This architecture is a standard choice for continuous control tasks and provides sufficient capacity for our problem.
- **Learning rates:** The actor and critic networks use Adam optimisers with initial learning rates of 3×10^{-4} . The entropy temperature α is also automatically tuned by RLlib or adjusted via its own learning rate (1×10^{-3} in our setup).
- **Training batch sizes:** We set the SAC learner to use a rollout fragment length of 100 steps and a train batch size of 2000 steps for each gradient update. This means the agent’s replay buffer collects 2000 time steps of interaction data before each learning iteration.
- **Replay buffer and off-policy updates:** The agent leverages a replay buffer (maintained internally by RLlib) to reuse past experiences. Off-policy training allows the agent to continue learning from older trajectories, which is helpful if certain critical scenarios (like approaching a station) occur infrequently.
- **Stop criteria and checkpoints:** We impose a training time limit of 3600 seconds (one hour) for practical reasons. In a full-scale experiment we would allow significantly longer training, but within our project constraints this limit ensures the experiment terminates. We also configure periodic checkpoints. Our code can detect if a previous run’s checkpoint exists and resume training from it, which is useful to extend training across multiple sessions or recover from interruptions.

Training is executed using Ray Tune’s `Tuner`. If a checkpoint directory from a previous training session is present, our script will restore from it (with an option to resume even if the last run ended with an error). Otherwise, it initiates a fresh training run. We gave each training experiment a unique name (reflecting the number of parallel environments) and stored results (including models and logs) under a dedicated directory.

In summary, the code initialises the Ray runtime, sets up the environment and SAC configuration, and then starts the training loop. Throughout training, RLlib logs performance metrics such as average reward per episode, which we monitored to evaluate learning progress.

The pseudocode below summarises the interaction between the RL agent and the simulation environment:

Algorithm 1 Environment-Agent Interaction Loop

```

1: while not done do
2:   action  $\leftarrow$  agent.decideAction(state)
3:   sendActionToSimulator(action)
4:   state  $\leftarrow$  getSimulatorObservation()
5:   reward  $\leftarrow$  computeReward(state)
6:   done  $\leftarrow$  checkDoneCondition(state)
7:   agent.update(state, reward)
8: end while

```

The following pseudocode illustrates our reward function:

Algorithm 2 Reward Calculation

```

1: overspeedPenalty  $\leftarrow$   $\max(\text{speed} - \text{speedLimit}, 0)^2$ 
2: if nearStation and braking and speed = 0 then
3:   reward  $\leftarrow$  reward + stationStopBonus
4: else
5:   reward  $\leftarrow$  reward - brakingPenalty
6: end if
7: reward  $\leftarrow$  reward - overspeedPenalty
8: reward  $\leftarrow$  reward + distanceTravelledIfNoOverspeeding
9: return reward

```

The SAC implementation is based on RLlib. Key configurations from `main.py` include:

Listing 1: Excerpt from `main.py` SAC Configuration

```
config = {
    "env": "OpenRails-v0",
    "num_workers": 1,
    "framework": "torch",
    "rollout_fragment_length": 100,
    "train_batch_size": 2000,
    "Q_model": {"fcnet_hiddens": [256, 256]},
    "policy_model": {"fcnet_hiddens": [256, 256]},
    "optimization": {
        "actor_learning_rate": 3e-4,
        "critic_learning_rate": 3e-4,
        "entropy_learning_rate": 1e-3
    },
    "stop": {"time_total_s": 3600}
}
```

The agent controls throttle and brake actions communicated via JSON requests to Open Rails. The observation function processes data logged by the simulator:

Listing 2: Observation Processing from `OpenRailsEnv.py`

```
def get_observation():
    data = parse_simulator_log()
    obs = [
        normalize(data.time_elapsed),
        normalize(data.throttle),
        normalize(data.brake_force),
        normalize(data.speed),
        normalize(data.speed_limit),
        normalize(data.distance_to_next_station)
    ]
    return obs
```

3.3.3 Changes in the Agent

Over the course of the project, we iteratively refined the agent and environment configuration, resulting in three main versions of the agent (v1, v2, v3):

- **Agent v1:** The initial version of the agent used a comprehensive reward function accounting for punctuality, speed limit compliance, passenger comfort (jerk minimisation), and energy efficiency. However, balancing these multiple reward components proved difficult with limited tuning experience. The v1 agent converged to an overly conservative policy, prioritising speed limit adherence above all else (to avoid heavy penalties) and thus sometimes sacrificing schedule adherence and efficiency.
- **Agent v2:** In the second iteration, we simplified the reward function to focus primarily on speed optimisation and schedule adherence, temporarily removing or down-weighting the energy and comfort terms. The rationale was to allow the agent to first learn to drive fast yet within speed limits and on time, then introduce additional objectives later. This led to a less conservative policy: the agent explored higher speeds when appropriate, although it still had to relearn aspects of braking and acceleration efficiently. We also introduced a multitude of small changes to stabilize learning such as:
 1. Normalized all the observation values on a range from 0 to 1.
 2. Reduced the learning rate.
 3. The agent can now dynamically receive the information of how much distance is left till the next speed limit.
- **Agent v3:** For the third version, we introduced further refinements based on insights from v1 and v2. One significant adjustment was limiting the maximum brake command to about 33% of full scale, preventing the agent of extremely hard braking. Increased the agents Network Depth and Width to 512x3 from 256x2. By doubling the hidden units and adding an extra layer to give the policy and value networks more representational capacity, making it easier to learn the complex mappings needed for optimal speed profiles. Lastly we increased the discount factor to 0.995 from 0.99, to emphasize longterm rewards. The v3 agent represents our most balanced policy, with improved adherence to both schedule and safety.

Each new agent version was trained from scratch (not fine-tuned from a previous model) due to the significant changes in reward structure. Therefore, the performance differences observed stem from the different learning objectives rather than incremental learning on the same objective.

Table 1: Overview of differences in the versions of the agent

	Version 1	Version 2	Version 3
Reward function aims	Punctuality, speed limit compliance, jerk and energy efficiency	Punctuality and speed limit	Punctuality and speed limit
Learning rate	Very High	High	Standard
Dynamically receive distance to next speed limit	No	Yes	Yes
Maximum break %	50%	50%	33%
Normalized observation values	No	Yes	Yes
Network Depth and Width	256x2	256x2	512x3
Discount factor	0.99	0.99	0.995

Bellow follows two listings showing the different reward funtions for V1 compared to V2 and V3 in detail.

Listing 3: Main part of reward function for V1

```
#weight variables
alpha = 1*0
beta = 1.5
gamma = 1/(1e6)
omega = 1

#Main variables
previous_acceleration = self.prev_acceleration.popleft()
self.prev_acceleration.append(acceleration)

jerk = abs(acceleration - previous_acceleration)

over_speed_limit = (speed - speed_limit)
if over_speed_limit < 0:
    over_speed_limit = 0

if distance < 3000 and brake_value > 0:
    brake_penalty = brake_value * 100

#Main Reward function
R = -(alpha * jerk) -(over_speed_limit ** beta) -(gamma *
    motive_force) - brake_penalty

reward = R + (distance - self.distance_driven) * omega

self.distance_driven = distance
```

Listing 4: Main part of reward function for V2 and v3

```
#weight variables
alpha = 0
beta = 1.5
gamma = 0
omega = speed/100 + 1

#Main variables
previous_acceleration = self.prev_acceleration.popleft()
self.prev_acceleration.append(acceleration)

jerk = abs(acceleration - previous_acceleration)

over_speed_limit = (speed - speed_limit)
if over_speed_limit < 0:
    over_speed_limit = 0

#Main Reward function
brake_penalty = 0
if distance_to_next_station > 500 and speed < 5 :
    brake_penalty = brake_value*300+100

R = -(alpha * jerk) - (over_speed_limit ** beta) - (gamma *
    motive_force) - brake_penalty

reward = R
if over_speed_limit == 0:
    reward += (distance - self.distance_driven) * omega
self.distance_driven = distance
```

3.4 Route Description and Scenario

All experiments were conducted on a specific route segment: the main line between Trollhättan and Gothenburg in Sweden. This route was chosen because it provides a realistic mix of track conditions, including multiple speed limit changes, and gradients. The route length (roughly 73 km) and timetable constraints (40-50 min) are representative of a medium-distance intercity service.

To initialise the simulation consistently, we created a scenario in Open Rails corresponding to a train departing Trollhättan and heading towards Gothenburg under normal conditions (no extraordinary delays or disruptions beyond those potentially introduced by the agent’s driving). The train model used was a standard electric passenger train with characteristics (mass, power, braking profile) typical for that service, the SJ X2000.

A baseline human-driving profile was not directly recorded; however, we established a reference travel time based on a rule-of-thumb driving pattern. It is generally observed that experienced drivers often operate at around 80–85% of the posted speed limit to maintain a safety margin. Using this assumption, we estimated a reasonable journey time: driving at 80–85% of each segment’s speed limit from start to finish yields an expected travel time in the range of approximately 43 to 46 minutes. This range serves as a benchmark for the agent’s performance. An agent consistently taking much longer than 46 minutes would be considered too conservative, whereas an agent significantly faster would likely be violating speed limits or driving unsustainably.

3.5 Human Driving Profile (Baseline)

As a point of comparison, we defined a notional human driver profile for the route. In absence of actual driver data, the baseline assumes an experienced driver who stays slightly below speed limits and anticipates stops smoothly:

- The driver targets roughly 80–85% of the posted speed limit in each section, accelerating up to that margin and coasting or lightly braking to maintain it.
- Well before reaching a lower speed limit zone or the final station, the driver begins coasting and then braking gently, aiming to avoid any overshoot of speed limits.
- This conservative approach results in a travel time of around 44 to 45 minutes for the route, which is safe and punctual under normal conditions.

This baseline provides context for evaluating the agent: if the RL agent can approach this travel time without violations, it is performing comparably to a human; if it can beat this time while respecting constraints, it would be an improvement.



Figure 1: Example speed profile of two humans driving the train, only the speed limit was taken under consideration (taking respectively 51:29 and 46:29 minutes)

3.6 Hardware and Runtime Setup

Training and simulations were run on two separate computers of moderate computing power (PC1: Ryzen 7 5800X 8-core CPU and RTX 4060 GPU. PC2: i7-8700K CPU @ 3.70GHz and GTX 1080 Ti). Each computer ran its own instance of the simulation and learning environment: one was primarily used for the initial long-duration training (agent v1, over roughly 3 months of intermittent training), and the other for subsequent shorter experiments (v2 for about 1 month, v3 for approximately 1–2 weeks). The machines were not networked for distributed training; instead, we treated each agent version’s training as an independent experiment on a single machine.

The use of two different machines (and thus two independently trained agents for v1 and v2) meant the resulting policies might differ not only due to reward changes but also due to the inherent randomness in training. Each training run consumed on the order of tens of hours of CPU time (accelerated by the fact that the simulation can run faster than real-time up to a point). No GPU acceleration was used, as the neural network sizes were small and CPU was sufficient for our needs.

All software (Open Rails and our Python RL code) ran on Windows 10. We used Python 3.9 with Ray RLLib version 2.x. Open Rails, being a graphical application, ran in the background with its rendering minimised to

reduce overhead, relying on its internal physics and our API calls for actual interaction.

4 Results

4.1 Training Curves

Over the course of training the first agent iteration (v1), we observed gradual improvements in performance as reflected in the reward curve.



Figure 2: Reward per metre of travel as a function of training episodes for the first agent iteration (v1).

Figure 2 shows the agent’s reward per metre travelled as training progresses. Initially, the agent’s performance was poor due to random exploration and an overly complex reward signal. Over successive runs, the average reward increased, indicating the agent tended to be more efficient at meeting the objectives.

However, learning was not monolithic. The v1 agent faced significant challenges early on as we attempted to balance contradictory objectives (maximising speed while minimising overspeed and jerk, simultaneously with energy conservation). The reward curve for v1 shows a lot of variance and a slow upward trend, reflecting the agent’s struggle to find a good compromise policy.

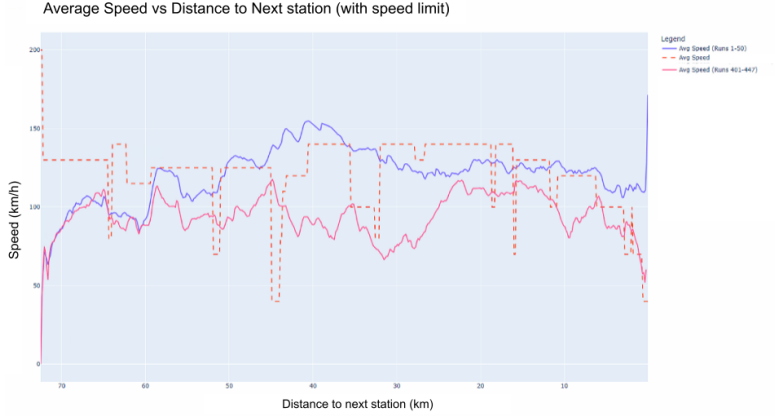


Figure 3: Agent v1’s speed profiles across the route: first 50 runs (blue) vs last 50 runs (red) of training, with track speed limit shown in orange.

We analysed the agent’s behaviour at different stages of v1’s training. Figure 3 contrasts the agent’s speed profiles over distance during its first 50 episodes versus the last 50 episodes of the training run. In early episodes, the agent frequently exceeded speed limits (blue traces above the orange limit line) and incurred heavy penalties, but by the later episodes (red traces), it learned to stay below the speed limit line almost all the time. This improvement in rule compliance came at the expense of a lower average speed; the agent became very cautious, often not accelerating up to the limit even where it could.

Given this outcome, we adjusted our approach for the second training iteration. Noticing that the agent had difficulty handling multiple objectives, we narrowed the focus for v2. Jerk minimisation and explicit energy terms were removed from the reward, so the agent v2 primarily tried to run as fast as possible while avoiding speed infractions and staying on schedule. This led to a clearer learning signal.

With agent v2, training was much more stable. The reward per episode increased faster and with less variance. We also observed that v2 quickly learned to consistently obey speed limits (much sooner than v1 did) as evidenced by near-zero overspeed penalties after sufficient training.

In summary, the training phase revealed that simplifying the learning problem (as done from v1 to v2) can significantly improve learning speed and policy quality. It also highlighted the importance of reward design. Initially, the agent was being “pulled” in too many directions by the reward

function, which made learning inefficient.

To further diagnose the agent’s limitations, we examined why even after extensive training the v1 agent did not achieve near-optimal times. We found that the agent rarely engaged in anticipatory control. For example, it would not gradually accelerate to just below a forthcoming higher speed limit before actually reaching it, nor would it smoothly decelerate well in advance of lower speed zones. Instead, its control was reactive: it only started slowing down once it hit a speed restriction, often braking harder than an optimal strategy would require.

These observations influenced our subsequent improvements and indicated that adding more foresight (e.g., observations that inform the agent of upcoming track changes rather than relying on the agent calculating where next track changes are), combined with increasing its neural network size, could help the agent plan better.

4.2 Energy Consumption Analysis

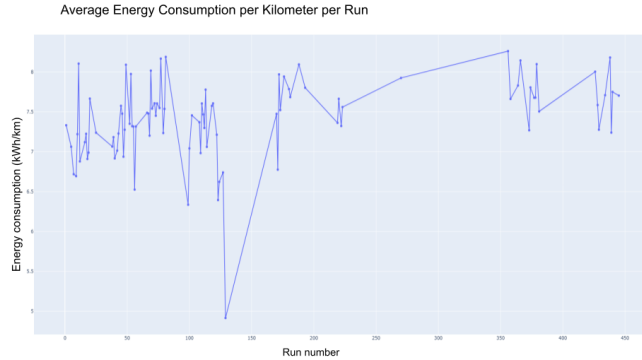


Figure 4: Agent v1’s average energy consumed per km, calculated over runs where at least 95% of the route was completed. The agent’s aggressive throttle use is reflected in the energy consumption, indicating that without explicit incentives, the RL agent does not naturally conserve energy. The variability also points to inconsistent driving patterns across episodes.

An important metric is how well the agent manages energy. Figure 4 illustrates the average energy consumption per kilometre for the agent’s trajectories (for agent v1, considering only episodes where the agent completed

at least 95% of the route to filter out aborted runs). We expected that an agent optimising travel time and obeying speed limits, but without an explicit energy term, might have used more energy than necessary (e.g., by heavy acceleration and braking).

Indeed, the results show that our current agent does not significantly prioritise energy efficiency. The v1 and v2 agents, while adhering to speed limits, often apply maximum throttle to make up time and then brake to stay under limits, which is not energy-optimal. The energy usage was higher than what a skilled human driver or an energy-aware optimal control solution would achieve. For agent v3, we started to address this by implicitly penalising extreme throttle changes (through the smoother braking constraint and minor reward tweaks). However, even v3’s behaviour suggests room for improvement in energy management.

The relatively high energy consumption likely stems from the reward structure lacking a strong incentive for saving energy. With further reward tuning or multi-objective training, the agent could learn to trade off a small increase in travel time for a large energy saving (for instance, coasting more often). This trade-off was not realised in the current iteration.

4.3 Schedule Adherence

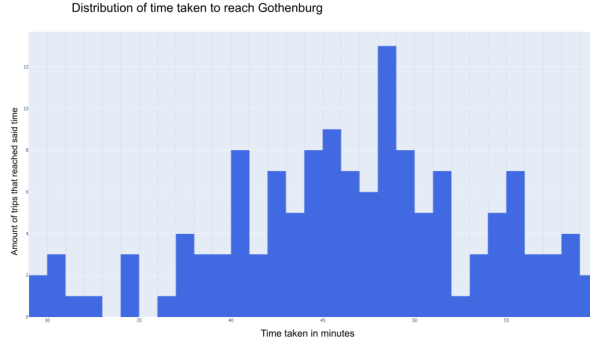


Figure 5: Distribution of journey completion times for agent v3. The average is 46:32 (min:sec) with a standard deviation of 6:56. Dashed lines indicate theoretical times for driving at 85% (43:24) and 80% (46:06) of speed limits. The agent’s performance is within a few minutes of human-like driving times, indicating reasonable schedule adherence.

From the perspective of timetable adherence, the agent’s performance is encouraging. Figure 5 shows the distribution of total journey times for the agent v3 over many episodes. The average trip time was approximately 46 minutes and 32 seconds, with a standard deviation of about 6 minutes and 56 seconds. This variance arises from the stochastic nature of the policy and minor random differences in simulation conditions.

We compared these times to our theoretical targets: driving at 85% of speed limits yields about 43:24 (min:sec), and at 80% yields 46:06. The agent’s average of 46:32 is just slightly above the upper bound of this range. This means the agent, on average, is close to what a reasonably efficient human driver might achieve, albeit a bit slower. Some of the agent’s runs did approach 43–44 minutes (when it pushed harder), while others were slower (when it was more cautious).

Importantly, the agent consistently avoided being significantly late. In none of the evaluated episodes did the agent exceed, say, 50 minutes travel time (unless it stopped learning early due to our time cutoff). This indicates that even though the agent is conservative, it still meets schedule requirements with some buffer.

One might wonder if the agent could further reduce its average time. Likely yes, with improved anticipation and perhaps a slightly more aggressive tuning, the agent could shave off another minute or two on average. Agent v3 still exhibits some inefficiencies; addressing those could bring its mean closer to 44–45 minutes without breaking rules.

4.4 Driving Profiles

Figure 6, 7, and 8 show the best (most efficient) speed curves generated by agents v1, v2, and v3 respectively, along with their total travel times. These illustrate how the driving strategy evolved.



Figure 6: Speed profile of best run from Agent v1 (red) with speed limits (black). Total time = 39:20. Agent v1 achieved a very fast time but frequently exceeded speed limits (red above black), which is not acceptable in practice.

Agent v1's profile (Figure 6) shows a total time of 39:20, but this was achieved with significant overspeeding. The speed curve for v1 crosses above the permitted limits multiple times, meaning the agent violated constraints to get that time. Such behaviour was heavily penalised in training, so while v1 managed a very fast run, it was not typical or acceptable.

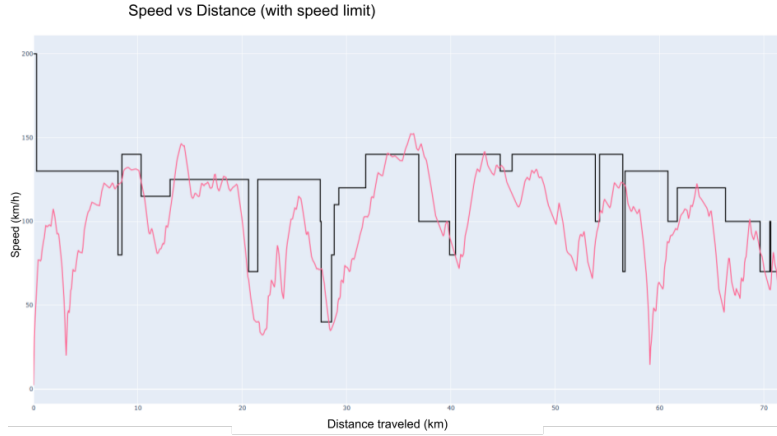


Figure 7: Speed profile of a typical run from Agent v2 (red) with speed limits (black). Total time = 58:40. Agent v2 follows speed limits better, overspeeding less. But it also occasionally comes to almost a deadstop, resulting in a significantly longer travel time.

Agent v2's profile (Figure 7) is much slower at 58:40. Here the agent obeyed all speed limits (no speeding, as the speed stays under the limit line) but was extremely cautious throughout, often leaving a large margin below the speed limit. The result is a punctual but inefficient run, exemplifying the conservative policy that v1 had converged to. v2 essentially represented that extreme safe driving once we removed the time pressure.

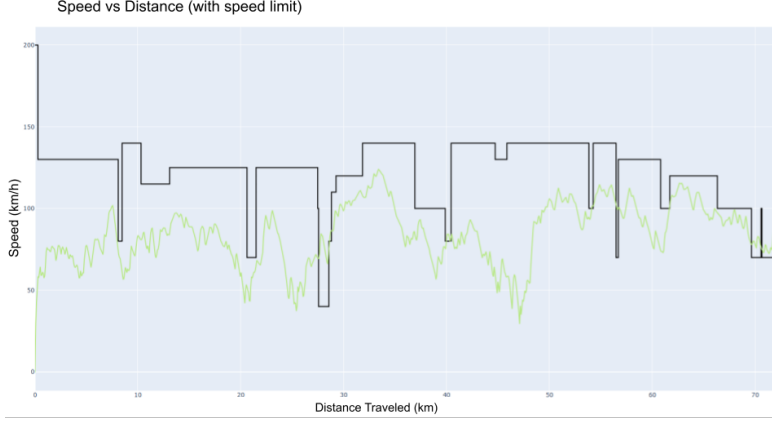


Figure 8: Speed profile of an optimal run from Agent v3 (green) with speed limits (black). Total time = 54:39. Agent v3 adheres the best to all of the speed limits and utilises the allowable speed more efficiently than v2, though it still leaves significant margin before certain restrictions.

Agent v3’s profile (Figure 8) improved to 54:39, better than v2 while still adhering to speed limits. The v3 speed curve is closer to the limit line than v2’s, though it still leaves some gap where the agent didn’t fully exploit the allowed speed. Notably, v3 incurred virtually no overspeed instances and represents the best balance among the three versions: it never breaks rules yet makes decent progress.

Table 2: Relative overspeeding instances for each agent version (v3 = 100% baseline). Lower values indicate fewer or less severe speed violations.

Agent	v1	v2	v3
Relative overspeed incidents (%)	681.46%	157.55%	100%

Table 2 quantifies the relative overspeeding frequency of each agent (normalised so v3 = 100%). Agent v1 incurred about 681% of v3’s overspeed incidents, and agent v2 around 158%. This confirms v3 had the least overspeeding (essentially zero beyond trivial amounts), v2 also very safe, and v1 very unsafe in comparison.

From these comparisons, we conclude that agent v3 achieved the best compliance-efficiency trade-off among the versions. It drastically reduced

speed violations relative to v1, and it improved travel time relative to v2. There remains a gap between v3's 54:39 and the human baseline (45 minutes). Bridging this gap will likely require further tuning and providing the agent with more foresight to use the available headroom effectively.

Overall, the trajectory profiles underscore the progress: through hybrid modelling and iterative refinement, we shaped an RL policy that drives a train in a rule-abiding manner. The remaining conservatism in the agent's behaviour is not necessarily negative, it ensures safety, but it shows the hybrid approach did not yet fully reach the optimal potential. Still, it provides a foundation for future improvements (e.g., adding preview inputs for upcoming speed changes or adjusting exploration vs exploitation balance).

5 Discussion

5.1 Interpretation of Learned Policy

The learned policy from our final agent (v3) demonstrates a clear emphasis on safety and regulatory compliance, as evidenced by its strict adherence to speed limits throughout the route. During early training, the agent frequently incurred substantial penalties for overspeeding, which it gradually learned to avoid by adopting a conservative driving style. Specifically, the agent tends to brake early and more than necessary when approaching speed limit reductions or stops. This behaviour reflects the high weight given to the speed-limit penalty in the reward function, biasing the agent toward avoiding infractions even at the expense of driving speed.

However, the policy’s limitations are apparent in how it handles transitions in allowed speed. We observe that the agent’s deceleration and acceleration patterns are somewhat abrupt; it often over-brakes upon entering a lower speed zone and then needs to re-accelerate, rather than executing a smooth, anticipatory slowdown. This reactive control indicates that the state representation and reward shaping did not sufficiently encourage foresight. The agent is essentially reacting to the immediate speed limit rather than planning ahead for upcoming changes. In part, this is due to the Markovian nature of the RL state – if the agent is not given information about an upcoming speed limit change, it can only learn a reactive strategy. Our current state inputs did include distance to the next station, but not explicitly the distance to the next speed limit change; adding that could improve this aspect.

Another contributing factor is SAC’s inherent exploration: the stochastic policy means that throttle and brake commands have some variability (noise) around the learned mean action. In practice, this manifests as small oscillations around the target speed. The agent doesn’t hold a perfectly steady speed; instead, it makes minor adjustments that can lead to a less smooth ride than an experienced human driver would provide. While these oscillations are not severe, they indicate that the agent hasn’t learned (or been incentivised) to minimise control variance (which relates to passenger comfort). A future refinement could involve adding a small penalty on jerk or excessive control changes to smooth out throttle/brake usage once the agent has mastered the basics of speed control.

In summary, the agent’s policy strikes a balance between constraint satis-

faction and progress, but errs on the side of caution and tends to be reactive. It reliably avoids speeding and eventually reaches the destination, fulfilling the primary requirements. Yet it lacks the nuanced optimisation that a human driver or advanced controller might have – such as coasting precisely into a speed restriction or timing accelerations to fully exploit a higher speed section.

Interestingly, the conservative nature of the policy can be seen as the agent “playing it safe” in the face of uncertainty. Because overspeeding was heavily penalised and there was no direct penalty for being slightly slower than necessary (as long as it arrived before a schedule cutoff), the agent learned that it’s better to be safe than sorry. This is rational given the reward structure.

To push the agent towards a more aggressive yet still legal policy, one could gradually increase the emphasis on time efficiency in the reward once the agent has mastered safety. Essentially, a curriculum approach: first teach it not to speed (which we did), then teach it to optimise time within those safety bounds (which we partially did, but could be strengthened).

Finally, we note that the hybrid modelling approach (using a high-fidelity simulator within RL) succeeded in instilling realistic constraints in the agent’s behaviour. The agent cannot do physically impossible actions or ignore train dynamics because the environment enforces those. This is an advantage over purely theoretical models: the agent’s policy is directly tested on a realistic system. Any improvement in the agent’s driving strategy from this point should be transferable to the real world, since it was developed and evaluated in a simulator that closely mimics real train behaviour.

5.2 Limitations

This project’s results are preliminary and come with several important limitations:

1. **Reward Tuning Complexity:** Balancing the multiple terms in our composite reward (speed compliance, schedule adherence, energy efficiency, and comfort) proved challenging. Early in the project, limited experience with reward shaping led to suboptimal weightings. As a result, the agent’s initial policy converged to overly conservative behaviours and failed to explore more optimal speed profiles. We mitigated this by simplifying the reward in later iterations, but identifying

truly optimal weightings would likely require extensive experimentation or automated methods (e.g., reward meta-learning).

2. **Hyperparameter Sensitivity:** We found training stability and convergence speed to be quite sensitive to hyperparameters such as learning rates, batch size, entropy temperature, and network architecture. Our initial hyperparameter sweeps were cursory due to time constraints, meaning the agents might not have had the best possible settings. At times, the agent’s training plateaued or got stuck in a local minimum, further delaying progress. A more systematic hyperparameter search or use of auto-tuning algorithms could improve performance.
3. **Time Constraints on Training:** Training an SAC agent to high performance in this environment requires a large number of interactions. Our project timeline meant that if early training runs were misconfigured (e.g., poor reward weights or hyperparameters), we could not always afford to start over and let a new agent train for the required duration. Each full training run consumed a substantial amount of time. This limited our ability to iterate. In a real deployment or extended project, one would run training for many more episodes or use parallel simulations to gather more experience. The results we have, while promising, might have improved significantly with, say, double the training time or more powerful hardware.
4. **State Observation Limitations:** The state information fed to the agent did not explicitly include some potentially critical features, such as a preview of upcoming speed limit changes. As discussed, this forced the agent to be reactive. The simulator had all the track information, but our environment did not expose it to the agent to keep the state simple (we primarily gave current speed, distance to next stop, etc.). This is a limitation because the agent had to infer upcoming changes indirectly, which it did not do effectively. Exposing more route context (e.g., distance to next speed change) should help the agent plan ahead.
5. **Generalisation and Robustness:** We trained and tested on one specific route and train type. The agent’s policy may be overfitted to the patterns of that route (e.g., it might have learned precisely when to brake for that route’s stations). If deployed on a different route or if conditions change (like an unexpected delay requiring faster running,

or a temporary speed restriction), the agent might not adapt well. True robustness would require training on a variety of routes and scenarios or implementing continual learning mechanisms.

6. **No Explicit Comfort Consideration:** Although our initial reward had a jerk penalty, we dropped it in the final agent. Thus the agent was not directly penalised for hard braking or acceleration beyond their indirect effects on other metrics. A human driver values comfort, so an operational system would need to reincorporate such considerations. Our agent might occasionally brake or accelerate in ways that are slightly jerky, though the brake limit in v3 naturally limited extreme jerk. Still, this aspect was not fully addressed.

These limitations offer learning points for future work. For instance, augmenting the observation space and employing more extensive training can likely address many of them. To generalise the solution, training across multiple routes with domain randomisation (as planned) will be important. To incorporate comfort, we could add a mild penalty for large acceleration changes in the reward.

Despite the limitations, it is noteworthy that the agent never violated fundamental safety constraints after training (it stopped at the destination and obeyed speed limits in final iterations). The primary shortcomings were about optimality and efficiency, not safety. This is exactly the order of priorities one would want: first ensure safety, then optimise performance. Our hybrid model succeeded in ingraining safety; the fine-tuning for efficiency can come next.

5.3 Implications

Our results suggest that, while full real-time autonomy for train operation (i.e., replacing a human driver entirely) remains a long-term goal, there are two promising pathways for leveraging this technology in practical settings:

1. **Pre-Computed Optimal Profiles:** For routes with relatively static conditions and fixed schedules, our framework could be used offline to generate optimised speed profiles in advance. These speed curves, computed by the agent (or an improved version) via simulation, can then be provided to human drivers as guidance. Essentially, drivers would receive a recommended speed timetable (perhaps as a sequence of target speeds at certain locations)

that they can follow to ensure efficient and punctual operation. This could improve energy efficiency or reduce equipment wear without requiring any AI decisions on the fly. It is a way to use the agent’s insights in an advisory capacity.

2. AI-Assisted Advisory Systems: By exploiting accelerated simulation, an RL agent could run in parallel with actual operations to evaluate current conditions and suggest adjustments. For example, if a train is unexpectedly delayed, an agent could quickly simulate scenarios and advise the driver on how much to accelerate in the next segment to recover time while still saving energy. Such a “digital co-driver” would operate in a supervisory role, providing situational advice while leaving final authority with the human operator. Early studies on AI-based driving assistance outline architectures where a digital adviser module filters relevant information and presents concise, context-aware recommendations to the driver [15].

Experienced drivers might gain less from such a system in normal running, but less experienced drivers or unusual conditions could benefit. Moreover, by retaining the human in control, we adhere to emerging Grades of Automation (GoA) frameworks that emphasise human–AI collaboration over full autonomy [16].

Implementing an advisory system requires careful human factors consideration. The AI’s advice must be clear and trustworthy. Also, any AI system must undergo rigorous safety validation (especially in rail) before deployment. One approach is to introduce it gradually: perhaps first as a suggestion tool for driver trainers, then as a non-binding advisory for actual drivers, and eventually as a standard tool once proven reliable.

In terms of automation levels, our approach currently aligns with GoA 2 or 3 (automated systems with driver supervision). We are far from GoA 4 (unattended operation), but these intermediate steps could significantly enhance operations. For instance, even a 5–10% improvement in energy efficiency across a rail network from better driving could save substantial costs and emissions, making such a system valuable.

Ultimately, the hybrid approach demonstrates a path not only toward autonomous trains, but also toward smarter train operations in general. It points to a future where trains run more efficiently by leveraging AI, whether autonomously or as driver aids, leading to energy savings, improved timetable adherence, and potentially smoother rides, all while maintaining the high safety standards of rail transport.

6 Conclusion

6.1 Summary of Findings

In this thesis, we explored a hybrid modelling approach that combines machine learning with physics-based simulation to optimise a train’s speed profile. Using the Soft Actor-Critic reinforcement learning algorithm interacting with the Open Rails simulator, we trained an agent to drive a virtual train between two stations under realistic operational constraints.

Our findings demonstrate that the approach is viable: the RL agent learned to operate the train in a manner that satisfies safety and timing requirements. Specifically, after training, the agent consistently obeys speed limits and is able to complete the journey within a reasonable time frame, showing the potential of machine learning to handle complex control tasks in rail transport. The agent’s performance in terms of schedule adherence and rule compliance is promising.

However, the results also highlight the significant computational effort required. The training process was intensive and time-consuming, and further training or fine-tuning would be necessary to reach human-expert-level performance. The current agent, while functional, remains somewhat conservative and suboptimal in travel time and energy efficiency. To deploy such models in real-world scenarios, additional research and development are needed to scale up training and refine the policy.

6.2 Contributions

This project’s contributions can be summarised as follows:

- We developed a novel integration of a high-fidelity open-source train simulator (Open Rails) with a state-of-the-art reinforcement learning algorithm (SAC), demonstrating a framework for hybrid modelling in train speed optimisation.
- Through iterative experimentation (agents v1, v2, v3), we offered insights into reward shaping and training strategies for complex multi-objective problems. In particular, our experience underscores the importance of gradually increasing problem complexity (i.e., start with simpler objectives, then add complexity) when training an RL agent for a difficult control task.

- We quantitatively evaluated the trained agents’ performance in terms of adherence to speed limits, schedule, and energy use, providing a baseline for what current RL methods can achieve on a realistic train driving task.
- By discussing practical implications, we bridged the gap between theoretical machine learning results and real-world rail operations, suggesting how such technology could be applied (as offline optimisation tools or real-time driver assistants) rather than assuming immediate full automation.

The work was a collaboration between Adrian Nagorka and Joel Håkansson, with both authors contributing to the literature review, system development, experiments, and analysis. In particular, one author focused on setting up the simulation environment and ensuring its realism, while the other concentrated on the machine learning implementation and parameter tuning. Writing and analysis were joint efforts, with frequent discussions to interpret the agent’s behaviour and troubleshoot training issues.

6.3 Future Work

Building on this investigation, several key directions emerge for future research:

1. **Distributed, Parallel Training:** A straightforward way to improve the agent’s performance is to run many simulations in parallel to expose the agent to a wider variety of experiences in less time. We plan to deploy the environment on a computing cluster where dozens of Open Rails instances can run concurrently. Using a distributed RL approach, a central SAC agent could aggregate experience from all instances. This would dramatically increase the volume of training data and could lead to faster convergence and a more robust policy. Technical challenges include orchestrating many simulator processes and ensuring they don’t interfere, but our code structure is amenable to scaling.
2. **Domain Randomisation and Multi-Route Generalisation:** To ensure the agent’s policy is not overfitted to a single route, future training will incorporate domain randomisation. We will train the agent on multiple route profiles (with different station spacings, gradients, and

speed patterns) and introduce random variations in conditions (e.g., signal delays, weather effects). This would help the agent learn general principles of efficient train driving that transfer to unseen conditions. Preliminary steps include adding features for upcoming track changes to the state, and training on routes in both directions to account for gradient differences.

3. **Online and Continual Learning:** Ultimately, we envision deploying the agent in a setting where it can continue learning from actual operations. If integrated into a driver assistance system, the agent could receive real-world data of train runs and adjust its policy over time (via continual reinforcement learning). This poses safety challenges (we wouldn't want the agent to try untested strategies on live trains), but one approach is to use simulation as a sandbox for policy updates: the agent collects real data, retrains or fine-tunes in simulation with those data, and if the updated policy shows improvement without regression in simulation, it is then used in production. This continual improvement loop would allow adaptation to changes like new timetables or rolling stock.
4. **Human-in-the-Loop Validation:** As we move toward practical implementation, involving human drivers in testing is crucial. We plan to integrate the trained agent into a prototype driver assistance interface and conduct user studies. Experienced train drivers would use the system in a simulator to provide feedback. Key aspects to evaluate include: Is the AI's advice understandable and acceptable? Does following the AI-generated profile feel natural? Such studies will refine the interface and build trust. Additionally, driver feedback can be used to adjust the model's priorities (for example, if drivers find certain control patterns uncomfortable, we can penalise those more).

In conclusion, this thesis lays the groundwork for a new approach to optimising train operations using hybrid modelling. The encouraging results obtained with limited time and resources indicate that with further development, such techniques could significantly contribute to the efficiency and automation of railway transportation. By proceeding carefully, improving the algorithms, expanding the scenarios, and working closely with human experts, we aim to move step by step toward train control systems that are smarter, safer, and more efficient than what is possible today.

References

- [1] X. Lai, Z. Yang, J. Xie, and Y. Liu, “Reinforcement learning in transportation research: Frontiers and future directions,” *Multimodal Transportation*, vol. 3, p. 100164, 2024.
- [2] W. Li and S. Ni, “Train timetabling with the general learning environment and multi-agent deep reinforcement learning,” *Transportation Research Part B: Methodological*, vol. 157, pp. 230–251, 2022.
- [3] C.-s. Ying, A. H. Chow, and K.-S. Chin, “An actor-critic deep reinforcement learning approach for metro train scheduling with rolling stock circulation under stochastic demand,” *Transportation Research Part B: Methodological*, vol. 140, pp. 210–235, 2020.
- [4] Y. He *et al.*, “Deep q-learning for real-time train scheduling on a single-track railway,” <https://arxiv.org/abs/2009.00433>, 2020, arXiv preprint arXiv:2009.00433.
- [5] W. Heineken, M. Richter, and T. Birth-Reichert, “Energy-efficient train driving based on optimal control theory,” *Energies*, vol. 16, no. 18, p. 6712, 2023.
- [6] Q. Zhou, J. Li, B. Shuai, H. Williams, Y. He, Z. Li, H. Xu, and F. Yan, “Multi-step reinforcement learning for model-free predictive energy management of an electrified off-highway vehicle,” *Applied Energy*, vol. 255, p. 113755, 2019.
- [7] Q. Guo, O. Angah, Z. Liu, and X. J. Ban, “Hybrid deep reinforcement learning based eco-driving for low-level connected and automated vehicles along signalized corridors,” *Transportation Research Part C: Emerging Technologies*, vol. 124, p. 102980, 2021.
- [8] A. B. Martinsen and A. M. Lekkas, “Curved path following with deep reinforcement learning: Results from three vessel models,” in *OCEANS 2018 MTS/IEEE Charleston*. IEEE, 2018.
- [9] H. Gao *et al.*, “Research on ato control method for urban rail based on deep reinforcement learning,” <https://www.researchgate.net/publication/367100102>, 2023, accessed April 29, 2025.

- [10] H. Pei and Z. Lan, “Optimization of high-speed train operation control based on soft actor-critic deep reinforcement learning algorithm,” *SSRN Electronic Journal*, 2023, preprint.
- [11] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, “Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor,” *International Conference on Machine Learning (ICML)*, pp. 1861–1870, 2018. [Online]. Available: <https://arxiv.org/abs/1801.01290>
- [12] Z. Wu, C. Gao, and T. Tang, “An optimal train speed profile planning method for induction motor traction system,” *Energies*, vol. 14, no. 16, p. 5153, 2021.
- [13] B. Chen, L. Zhang, G. Cheng, Y. Liu, and J. Chen, “A novel approach for train tracking in virtual coupling based on soft actor-critic,” *Actuators*, vol. 12, no. 12, p. 447, 2023.
- [14] J.-V. Merlevédé, S. Enjalbert, F. Vanderhaegen, and F. Flammini, “Towards artificial intelligence based rail driving assistance tool,” *EDP Sciences*, 2024, supervisory “Digital Adviser” and “Digital Co-Driver” modules for human-in-the-loop automation.
- [15] —, “Towards artificial intelligence based rail driving assistance tool,” in *ITM Web of Conferences, Proceedings of the MAIH 2024 Conference*, 2024, p. 03005.
- [16] Deutsche Bahn Digital Rail Germany, “Capacity and traffic management system: Deep reinforcement learning,” <https://digitale-schiene-deutschland.de/en/news/2022/capacity-and-traffic-management-system-Deep-Reinforcement-Learning>, 2022, accessed April 29, 2025.

7 Appendix

Listing 5: Main.py - the main function (V3)

```
# main.py

import os
os.environ["RAY_IGNORE_WINDOWS_JOB_OBJECT_ERROR"] = "1"
import psutil
import ray
from ray import tune
from ray.air.config import RunConfig
from ray.rllib.algorithms.sac import SAC
from ray.tune.registry import register_env
import OpenRailsEnv
import time
from ray.rllib.algorithms.callbacks import DefaultCallbacks

from ray.rllib.algorithms.sac import SAC

from gym.envs.registration import register

from ray.rllib.env.env_context import EnvContext

@ray.remote
class WorkerIndexCounter:
    def __init__(self):
        self.index = -1
    def get_next_index(self):
        i = self.index
        self.index += 1
        self.index %= 2
        return self.index

num_workers = 1
```

```

worker_index_counter = WorkerIndexCounter.remote()

def env_creator(config: EnvContext):

    worker_idx = config.worker_index

    port = 2149 + worker_idx
    base_url = f"http://localhost:{port}/API/CABCONTROLS"

    env = OpenRailsEnv.OpenRailsEnv(config)
    env.api_url = base_url
    env.worker_idx = worker_idx
    return env

# Register the environment with Ray Tune.
register_env("OpenRails-v0", env_creator)

def main():
    """
    This function sets up and runs a Ray Tune experiment to train an
    RLlib SAC agent
    on the custom 'OpenRails-v0' environment.
    """

    # 1) Initialize Ray's runtime.
    ray.init(ignore_reinit_error=True)

    # 2) Define the configuration dictionary for the RLlib algorithm
    (SAC).

    # - "env": The environment to train on (our custom one).
    # - "num_workers": How many parallel worker processes or "
    rollout" envs to use.
    # - "framework": "torch" or "tf" for PyTorch or TensorFlow,
    respectively.W
    # - "rollout_fragment_length": How many environment steps per
    sampling batch.
    # - "train_batch_size": How large a batch of experiences to
    train on each iteration.

```

```

# - "Q_model" and "policy_model": The neural network configs for
    SAC's Q-net and policy net.
# - "optimization": Learning rates for actor, critic, and
    entropy optimization in SAC.
# - "env_config": A dictionary with extra arguments for your
    environment's constructor (optional).
config = {
    "env": "OpenRails-v0",
    "num_workers": num_workers,
    "framework": "torch",
    "gamma": 0.995,
    "rollout_fragment_length": 50,
    "train_batch_size": 3000,
    "Q_model": {"fcnet_hiddens": [512, 512, 512]},
    "policy_model": {"fcnet_hiddens": [512, 512, 512]},
    "optimization": {
        "actor_learning_rate": 3e-4,
        "critic_learning_rate": 3e-4,
        "entropy_learning_rate": 1e-3,
    },
    "env_config": {},
    "env_runner": {"sample_timeout_s": 250},
    "stop": {"time_total_s": 3600},
    "#callbacks": CSVLoggingCallbacks, # <-- This line adds your
        custom callbacks.
}

# 3) Run the training experiment using Ray Tune.

storage_path="C:/Users/Adrian/kex_project/results" # Where logs
    and checkpoints will be saved.

exp_name = f"save_states{num_workers}"
exp_dir = os.path.join(storage_path, exp_name)

if tune.Tuner.can_restore(exp_dir):
    tuner = tune.Tuner.restore(
        exp_dir,

```

```

        trainable=SAC,
        resume_errorred=True
    )
else:
    tuner = tune.Tuner(
        SAC,
        param_space=config,
        run_config=RunConfig(
            storage_path=storage_path,
            name=exp_name
        )
    )

    tuner.fit()

if __name__ == "__main__":
    main()

```

Listing 6: OpenRailsEnv.py - the enviroment code (V3)

```

import gymnasium as gym
import numpy as np
import time
import pyautogui
import os
import datetime
import psutil
from collections import deque
import subprocess
import requests
import ray
import sys

class OpenRailsEnv(gym.Env):
    metadata = {"render_modes": []}

    def __init__(self, config=None):
        super(OpenRailsEnv, self).__init__()
        self.next_station = None
        self.current_error_timer = 100
        self.time_limit = 60 * 60 # 60 minutes in seconds

```

```

self.global_store = config.get("global_store", None)

self.time_speed = 8
self.over_speed_limit = False

self.error_count = 0
self.error = False

self.last_line = None
self.no_update_count = 0
self.no_update_threshold = 5

self.stop = False
self.stop_time = time.monotonic()

self.start_time = time.monotonic()

self.throttle = 0
self.brake_value = 0
self.speed = 0

self.reward = 0
self.first_time = True

self.current_obs = [0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,]

# Using a deque with a fixed maxlen for previous
# acceleration values.
self.prev_acceleration = deque([0], maxlen=1)

self.next_station_name = ""
self.distance_driven = 0

# deques containing speed limit distances and corresponding
# limits.
self.speed_limit_milestones = deque([282.0, 8088.0, 8463.0,
    10306.0, 13082.0, 20604.0, 21486.0, 27476.0, 27561.0,
    28560.0, 28814.0, 29256.0, 31834.0, 36918.0, 39874.0,
    40443.0, 44726.0, 45866.0, 53838.0, 54250.0, 56474.0,
    56688.0, 60801.0, 61697.0, 66319.0, 69635.0, 70567.0,

```

```

        70649.0, 71856.0])
self.speed_limit = deque([130.0, 80.0, 140.0, 115.0, 125.0,
    70.0, 125.0, 100.0, 40.0, 80.0, 110.0, 120.0, 140.0,
    100.0, 80.0, 140.0, 130.0, 140.0, 100.0, 140.0, 70.0,
    130.0, 100.0, 120.0, 100.0, 70.0, 100.0, 70.0, 40.0])
self.next_speed_limit = self.speed_limit.popleft()
self.next_speed_limit_milestone = self.
    speed_limit_milestones.popleft()
self.next_speed_limit_dist = self.next_speed_limit_milestone

# Define action space: continuous throttle/brake between 0
    and 1. First entry is throttle and second is for breaks
self.action_space = gym.spaces.Box(low=0.0, high=1.0, shape
    =(2,), dtype=np.float32)

# Define observation space.
self.observation_space = gym.spaces.Box(low=-np.inf, high=np
    .inf, shape=(13,), dtype=np.float32)

self.config = config or {}

def url(self):
    return getattr(self, "api_url", "http://localhost:2150/API/
        CABCONTROLS")

def reset(self, seed=None, options=None):
    """Launch or reset the OR simulator and return the initial
        observation and an empty info dict."""
    # 1. Clear the file at the start of each episode.

    self.last_step_time = time.monotonic()
    self.release_brake_time = time.monotonic()

    # 2. Close any existing OR programs.
    if self.error:
        self.error = False
    else:
        self._close_or_process_if_needed()

    # 3. Start OR or reset scenario.

```

```

self._start_or_process()
self.first_time = False

self.error = False

self.speed_limit_milestones = deque([282.0, 8088.0, 8463.0,
    10306.0, 13082.0, 20604.0, 21486.0, 27476.0, 27561.0,
    28560.0, 28814.0, 29256.0, 31834.0, 36918.0, 39874.0,
    40443.0, 44726.0, 45866.0, 53838.0, 54250.0, 56474.0,
    56688.0, 60801.0, 61697.0, 66319.0, 69635.0, 70567.0,
    70649.0, 71856.0])
self.speed_limit = deque([130.0, 80.0, 140.0, 115.0, 125.0,
    70.0, 125.0, 100.0, 40.0, 80.0, 110.0, 120.0, 140.0,
    100.0, 80.0, 140.0, 130.0, 140.0, 100.0, 140.0, 70.0,
    130.0, 100.0, 120.0, 100.0, 70.0, 100.0, 70.0, 40.0])

self.distance_driven = 0

# 4. Wait for the initial observation.
initial_obs = self._get_observation_from_or()
self.current_obs = initial_obs
return initial_obs, {}

def step(self, action):
    """Apply action to OR, then read new state, compute reward,
    and check if done."""

    self._send_action_to_or(action)
    time.sleep((2.0*3)/self.time_speed)

    next_obs = self._get_observation_from_or()
    reward = self._compute_reward(next_obs)
    done = self._check_done_condition(next_obs, reward)
    if done == "error":
        done = True

    if self.error:
        self._close_or_process_if_needed()

```

```

        terminated = done
        truncated = False

        info = {"error": self.error}
        print("self_throttle_value:", self.throttle)
        print("self_brake_value:", self.brake_value)

        print("Reward:", self.reward)
        self.current_obs = next_obs

        return next_obs, reward, terminated, truncated, info

def _start_or_process(self):
    file_path = r"C:\OpenRailsCustom\Program\OpenRails.exe"
    #file_path = r"C:\Open Rails sorce\Program\OpenRails.exe"
    if self.worker_idx > 0:
        time.sleep(self.worker_idx * 10)
    print(self.worker_idx)

    self.reward = 0
    if self.first_time:
        pass

    # Launch OpenRails and create a new process group to allow
    # sending CTRL_C_EVENT.
    self.open_rails_process = subprocess.Popen(
        [file_path],
        creationflags=subprocess.CREATE_NEW_PROCESS_GROUP
    )

    time.sleep(30) #Menu screen delay

    time.sleep(30) #Loading screen delay
    payload = [
        {
            "TypeName": "THROTTLE",

```

```

        "Value": 1.0
    }
]
payload2 = [
    {
        "TypeName": "PANTOGRAPH",
        "Value": 1.0
    }
]
payload3 = [
    {
        "TypeName": "TRAIN_BRAKE",
        "Value": 0.0
    }
]
payload4 = [
    {
        "TypeName": "PAUSE",
        "Value": 0.0
    }
]
payload5 = [
    {
        "TypeName": "DIRECTION",
        "Value": 0.0
    }
]
payload6 = [
    {
        "TypeName": "TIME",
        "Value": float(self.time_speed)
    }
]
headers = {
    "Content-Type": "application/json" # se till att skicka
    rtt header
}

url = self.url()
try:

```

```

response = requests.post(url, json=payload, headers=
    headers)
print("Status_code:", response.status_code)
print("Response_text:", response.text)
response = requests.post(url, json=payload2, headers=
    headers)
print("Status_code:", response.status_code)
print("Response_text:", response.text)
response = requests.post(url, json=payload3, headers=
    headers)
print("Status_code:", response.status_code)
print("Response_text:", response.text)
response = requests.post(url, json=payload4, headers=
    headers)
print("Status_code:", response.status_code)
print("Response_text:", response.text)
response = requests.post(url, json=payload5, headers=
    headers)
print("Status_code:", response.status_code)
print("Response_text:", response.text)
response = requests.post(url, json=payload6, headers=
    headers)
print("Status_code:", response.status_code)
print("Response_text:", response.text)
except requests.exceptions.RequestException as e:
    print(f"Ett fel uppstod: {e}")

time.sleep(15*3/self.time_speed) #wait fo the train to start
    moving

def _close_or_process_if_needed(self):
    OR_PROCESSES = ["RunActivity.exe", "Menu.exe"]
    self.reward = 0
    for proc in psutil.process_iter(['pid', 'name']):
        if proc.info['name'] in OR_PROCESSES:
            print(f"Found {proc.info['name']} with PID={proc.info
                ['pid']}. Terminating it.")
            proc.kill()

```

```

        proc.wait()

#@staticmethod
def get_last_line(self):
    """Read the last line of a file."""
    try:
        response = requests.get(self.url())
        if response.status_code == 200:
            data = response.json() # Om svaret r i JSON-format
            print("data:␣", data)
            port = data["Port"]
        else:
            print("Fel␣kod:", response.status_code)

        file_path = f"C:/OpenRailsCustom/Program/Logs/
            PhysicalInfoDump{port}.csv"
        #file_path = f"C:/Open Rails sorce/Program/Logs/
            PhysicalInfoDump{port}.csv"
        print(file_path)
        with open(file_path, 'rb') as f:
            # Move to the end of the file
            f.seek(-2, os.SEEK_END)
            # Keep reading backwards until you find a newline
            character
            while f.read(1) != b'\n':
                f.seek(-2, os.SEEK_CUR)
            # Read the last line
            last_line = f.readline().decode('utf-8')
            return last_line.strip()
    except OSError:
        # Handle empty files or other issues
        return None

def _get_observation_from_or(self):
    # STEP 2: GATHER OBSERVATIONS

    # Some other state dimension:

    time.sleep(0.01)

```

```

last_line = self.get_last_line()
last_line = str(last_line)

if self.last_line is not None:
    if last_line == self.last_line:
        self.no_update_count += 1
    else:
        self.no_update_count = 0
self.last_line = last_line

variables = last_line.split(';')

try:
    time_str = variables[0].replace(',', '', '.')
    throttle = float(variables[2])
    motive_force = 0.0
    brake_force = float(variables[4])
    speed = float(variables[5].replace('km/h', '').replace(',', ''))
    speed_limit = float(variables[6].replace('km/h', '').replace(',', ''))
    acceleration = 0.0

    gradient = float(variables[9].replace(',', '', '.'))
    self.next_station_name = variables[10]
    distance_to_next_station = float(variables[11].replace(',', ''))
    brake_value = float(variables[12].replace(',', '', '.'))

    distance = 72502 - distance_to_next_station

    self.next_speed_limit_dist = self.
        next_speed_limit_milestone - distance

    self.throttle = throttle
    self.brake_value = brake_value

```

```

if speed > speed_limit:
    self.over_speed_limit = True
else:
    self.over_speed_limit = False

# when passing the change in speed limit, update the
# variables to the relevant metrics
if distance >= self.next_speed_limit_milestone:
    self.next_speed_limit = self.speed_limit.popleft()
    self.next_speed_limit_milestone = self.
        speed_limit_milestones.popleft()

time_obj = datetime.datetime.strptime(time_str, '%H:%M:%S
    .%f')
reference_time = datetime.datetime.strptime('12:00:00.00'
    , '%H:%M:%S.%f')
time_delta = time_obj - reference_time
total_seconds = time_delta.total_seconds()

if speed == 0 and not self.stop:
    self.stop_time = total_seconds
    self.stop = True

if self.stop and speed > 0:
    self.stop = False

obs = np.array([ #EVERYTHING HAS BEEN NOW NORMALIZED TO
    ISH 0 TO 1
    total_seconds / self.time_limit, #Time in seconds
    throttle / 100, # 0 to 100
    motive_force / 165728.0, # 0 to 165728.0
    brake_force / 67857.0, # 0 to 67857.0
    speed / 200, # 0 to 200
    speed_limit / 200, # 0 to 200
    acceleration / 100, # 0 to 100
    distance / 72502, # 0 to 72502
    gradient / 2.62, # 0 to 2.62
    distance_to_next_station / 72502, # 0 to 72502
    brake_value / 100, # 0 to 100

```

```

        self.next_speed_limit / 200, # 0 to 200
        self.next_speed_limit_dist / 7992.0 # 0 to 7992.0
    ], dtype=np.float32)
    self.speed = speed
    return obs
except:
    self.error = True
    self.speed = speed
    return self.current_obs

def _send_action_to_or(self, action):
    """Convert the RL action into throttle/brake keystrokes or
    commands."""

    throttle_value = action[0]
    brake_value = action[1]

    if (brake_value < 0.25 and self.speed < 5):
        brake_value = 0.0
    else:
        brake_value = brake_value/3

    print("Throttle_value:␣",throttle_value)
    print("brake_value:␣",brake_value)

    url = self.url()

    payload_throttle = [
        {
            "TypeName": "THROTTLE",
            "Value": float(throttle_value)
        }
    ]

    payload_brake = [
        {
            "TypeName": "TRAIN_BRAKE",
            "Value": float(brake_value)
        }
    ]

```

```

    }
]

headers = {
    "Content-Type": "application/json" # se till att skicka
    rtt header
}

try:
    response = requests.post(url, json=payload_throttle,
        headers=headers)
    response = requests.post(url, json=payload_brake, headers
        =headers)
except requests.exceptions.RequestException as e:
    print(f"Ett fel uppstod: {e}")

def first(self):
    self.first_time = True

def _compute_reward(self, obs):
    """
    STEP 3: COMPUTE REWARD
    - Typically negative energy usage or negative time if behind
      schedule, etc.
    - Possibly large negative if overshooting station or failing
      constraints.
    """

    #extract the relevant observation variables, also un
    normalize the values
    time_elapsed = obs[0] * self.time_limit
    throttle = obs[1] * 100
    motive_force = obs[2] * 165728.0
    brake_force = obs[3] * 67857.0
    speed = obs[4] * 200
    speed_limit = obs[5] * 200
    acceleration = obs[6] * 100
    distance = obs[7] * 72502
    gradient = obs[8] * 2.62

```

```

distance_to_next_station = obs[9] * 72502
brake_value = obs[10] * 100
next_speed_limit = obs[11] * 200
distance_to_next_speed_limit = obs[12] * 7992

print("Current_speed_limit:", speed_limit)
print("Current_speed:", speed)
print("Next_speed_limit:", next_speed_limit)
print("Distance_to_next_speed_limit:",
      distance_to_next_speed_limit)
print("Current_throttle:", throttle)
print("Current_Break_value:", brake_value)
#weight variables
alpha = 1*0
beta = 2
gamma = 1/(1e6) *0
#thetha = 0.25
omega = speed/100 + 1
station_reward = 1e5

#Main variables
previous_acceleration = self.prev_acceleration.popleft()
self.prev_acceleration.append(acceleration)

jerk = abs(acceleration - previous_acceleration) # jerk is
the absolute difference in acceleration change

over_speed_limit = (speed - speed_limit) # represents how
much over the speed limit the train is
if over_speed_limit < 0:
    over_speed_limit = 0

#Main Reward function

brake_penalty = 0
if distance_to_next_station > 500 and speed < 5 : #We want

```

```

        the brake penalty be lesser, but be applied across the
        run
        brake_penalty = brake_value*300+100

R = -(alpha * jerk) - (over_speed_limit ** beta) - (gamma *
    motive_force) - brake_penalty

reward = R
if over_speed_limit == 0: # only applied if we are not
    breaking the speed limit
    reward += (distance - self.distance_driven) * omega
self.distance_driven = distance

# Specific situation rewards and penaltis

if distance_to_next_station < 150 and over_speed_limit > 0:
    reward -= station_reward # massive penalty if we miss a
        station
    station_reward = 0

if distance_to_next_station < 30 and brake_value > 0 and
    speed == 0:
    reward += station_reward # big reward if we stop att a
        station
    station_reward = 0

if distance_to_next_speed_limit < 100 and over_speed_limit
    ==0:
    pass

self.reward += reward
return reward

def _check_done_condition(self, obs, reward):
    """
    STEP 4: CHECK IF EPISODE IS DONE
    - e.g. done if final station is reached or time limit
        exceeded.
    """

```

```

time_elapsed = obs[0] * self.time_limit
speed = obs[4] * 200
distance_to_next_station = obs[9] * 72574.0
brake_value = obs[10] * 100

if self.no_update_count >= self.no_update_threshold:
    print("Filuppdatering har stannat. Avslutar episod.")
    self.error = True
    return True

if (self.next_station_name == "Gteborg C" and
    distance_to_next_station < 30 and brake_value > 0 and
    speed == 0) or (time_elapsed > self.time_limit):
    done = True
elif self.error:
    done = True
elif self.stop and (time_elapsed - self.stop_time > 60*5):
    done = True
    self.stop = False
    reward -= distance_to_next_station
else:
    done = False

return done

```